

Jimma University

Faculty of Computing and Informatics

Computer Science Department

Web Programming

Jimma, Ethiopia.

PDO(PHP Data Object)

PDO overviews

- The PHP Data Objects (PDO) extension defines a lightweight, consistent interface for accessing databases in PHP.
- It will make database coding in PHP more secure, faster and portable (easier to change and many DBs are supported).
- Using prepared statements will help protect you from SQL injection so you do not need any own sanitize functions.



PDO

- Whichever database you're using, the steps to interact with a database are:
  - Connect to the database.
  - Send an SQL query that contains instructions for the database software.
  - If you retrieved data from the database, process the data.
  - Close the connection to the database.

Cont...

❖ The three main PDO classes are:

- PDO: represents a connection between PHP and a database server.
- PDOStatement: represents a prepared statement and, after the statement is executed, an associated result set.
  - ➢ It represents a prepared statement, as well as a returned result set.
  - ➢ It represents a prepared statement before it is executed, and the result after it is executed.
- PDOException: represents an error raised by PDO.

PDO

❖ For this course, we use a table called **users** that exists in a database called **demo**.

ID	FirstName	LastName	Sex	Telephone	Status
john232	John	Walter	M	4325456342	1
sam543	Samuel	David	M	5424256345	1
geo534	George	Graham	M	2634345643	0
ana979	Ana	Bush	F	3462737485	0
…	…	…	…	…	…

Connect to Database

- Connections are established by creating instances of the PDO class.
- The syntax.

public PDO::__construct (string $dsn [,string $username [,string $password [,array $options]]])

  - dsn - the Data Source Name, or DSN, contains the information required to connect to the database. Like driver – specifies the database driver used, host – specifies the database server to connect to, and dbname –specifies the database to connect to.
  - username - the database user name.
  - password - the database password.
  - options - a key=>value array of driver-specific connection options.

Example

❖ Example: connecting to different databases

```
$host = '127.0.0.1';
 $dbname = 'demo';
 $user = 'root';
 $pass = 123';
try {
# MySQL database
$dbh = new
PDO("mysql:host=$host;dbname=$dbname", $user,
$pass);
# Microsoft SQL Server
$dbh = new PDO("mssql:host=$host;
dbname=$dbname", $user, $pass);
    # Sybase database
$dbh = new PDO("sybase:host=$host; dbname=$dbname",
$user, $pass);
 # SQLite Database
 $dbh = new PDO("sqlite:my/database/path/database.db");
} catch(PDOException $e) {
    echo $e->getMessage();
}
```

- If there are any connection errors, a *PDOException* object will be thrown.
- To close the connection, you need to destroy the object by ensuring that all remaining references to it are deleted.
  - You do this by assigning **NULL** to the variable that holds the object.

Querying Database

- When using PDO to query a database, the function used to do so depends on the SQL statement you want to execute.
- There are two ways to run a query in PDO:
  - **PDO::query() – executes a SELECT query** and returns the result in the form of a **PDOStatement**. It is used only for SELECT queries.
  - **PDO::exec() – executes insert/update/delete queries** and returns the **number of rows affected.**
    - Use this method when you are executing a query that modifies the database (like insert, update or delete).

Cont...

- To execute a query, use:

  public PDOStatement PDO::query (string $statement);

- PDO::query() returns a PDOStatement object, or FALSE on failure.
- Example:

```
try {
   $dbh = new PDO("mysql:host=$hostname; dbname=$dbname",  $username, $password);
   $stmt = $dbh->query('SELECT * FROM users');
    …
   $dbh = null;
}catch(PDOException $e) {
   echo "Connection Error: ", $e->getMessage();
}
```

Fetching Data

- **Once a query is executed and a PDOStatement is returned**, the next step is to access the returned data row by row.
- PDOStatement provides several methods to do that.
  - The first way of fetching is using PDOStatement::fetch().
- This method is the most commonly used way of fetching rows.
  - mixed PDOStatement::fetch ([int $fetch_style [, int $cursor_orientation = PDO::FETCH_ORI_NEXT [, int $cursor_offset = 0]]])
- The parameters are:
- fetch_style - this controls how the next row will be returned to the caller.
  - This value must be one of the PDO::FETCH_* constants,
  - defaulting value of PDO::ATTR_DEFAULT_FETCH_MODE (which defaults to PDO::FETCH_BOTH).

Cont...

- The fetch style values are:
  - PDO::FETCH_ASSOC: returns the row as associative array indexed by column name
  - PDO::FETCH_BOTH: this is the default fetch style. It returns an array indexed by both column name and number.
  - PDO::FETCH_CLASS: assigns the values of your columns to properties of the named class. It will create the properties if matching properties do not exist.
  - PDO::FETCH_OBJ: returns the row as an anonymous object with property names that correspond to the column names
  - PDO::FETCH_LAZY: Combines PDO::FETCH_BOTH/PDO::FETCH_OBJ, creating the object variable names as they are used
  - PDO::FETCH_NUM: returns the row as an array indexed by column number

fetching rows as associative array and get the rows as number indexed array.

```
try{
 $db = new PDO('mysql:host=localhost;dbname=demo', 'root', 'vertrigo');
 $stmt = $db->query('SELECT * FROM users');
 while($row = $stmt->fetch(PDO::FETCH_ASSOC))
{
    echo $row['ID'];
    echo $row['firstname'];
    echo $row['lastname'];
    echo $row['sex'];
    echo $row['telephone']."<br>";
 }
 $db = null;
}catch(PDOException $e) {
   echo "Error during connection";
}
```

```
try{
  $db = new PDO('mysql:host=localhost;dbname=demo', 'root', 'vertrigo');
  $stmt = $db->query('SELECT * FROM users');
  while($row = $stmt->fetch(PDO::FETCH_NUM))
{
    echo $row[0];                 //ID
    echo $row[1];                 //firstname
    echo $row[2];                 //lastname
    echo $row[3];                 //sex
    echo $row[4], "<br>";         //telephone
  }
  $db = null;
}catch(PDOException $e) {
   echo "Error during connection";
}
```

returning it as an object instead of as an array.

```
try {
    $dbh = new PDO($dsn, "root", "vertrigo");
    $sql = "SELECT * FROM users";
    $stmt = $dbh->query($sql);
    while($obj = $stmt->fetch(PDO::FETCH_OBJ)) {
        echo $obj->ID;
        echo $obj->firstname;
        echo $obj->lastname;
        echo $obj->sex;
        echo $obj->telephone, "<br>";
    }
$dbh = null;
}catch(PDOException $e){
    echo $e->getMessage();
}
```

Fetching data—fetchObject

- **Fetching Object**
- PDOStatement allows you to fetch the next row as an object & return it.
- The syntax: PDOStatement::fetchObject ([string $class_name = "stdClass" [, array $ctor_args]])
  - This returns the next row as an object.
- This function is an alternative to PDOStatement::fetch() with PDO::FETCH_OBJ style.

```
$sql = 'SELECT * FROM users';
$stmt = $dbh->query($sql);
while ($obj = $stmt->fetchObject()) {
    echo $obj->ID. "\t". $obj->firstname. "\t" . $obj->lastname. "\t" . $obj->sex. "<br>";
}
```

Fetching a Single Column

- If you want to return only a single column from the row of a result set, you can use fetchColumn() method of PDOStatement.
- The syntax:

  mixed PDOStatement::fetchColumn ([int $column_number = 0]);

- It returns a single column from the next row of a result set.

```
$sth = $dbh->query("SELECT * from users");

print("Fetch the first column from the first row in the result set: <br>");

$result = $sth->fetchColumn();     //ID

print("name = $result<br>");

print("Fetch the second column from the second row in the result set:<br>");

$result = $sth->fetchColumn(1);    //firstname

print("colour = $result<br>");
```

Inserting Data

- PDO allows to insert data or modify data in the database.
- Once the connection is established, the INSERT query is accomplished using the PDO::exec method.
- PDO::exec() executes an SQL statement & returns the number of affected rows.
- The syntax is:

```
int PDO::exec ( string $statement )
```

```
$sql = "INSERT INTO users VALUES ('RV4321/06', 'Tom', 'Mike', 'male', '98766554', '1')";
$count = $dbh->exec($sql);
If($count >= 1)
    echo "Data inserted successfully";
else
    echo "Insertion failed";
```

Prepared Statements

- SQL statements need to be interpreted, checked for errors, analyzed & optimized, all before executing them with actual data.
- To protect against SQL injection, you prepare your SQL statement first, and then execute it with the data afterwards.
- A prepared statement is a precompiled SQL statement that can be executed multiple times by sending just the data to the server.
- PDO::prepare() prepares a statement for execution and returns a PDOStatement object.
  - PDOStatement PDO::prepare (string $statement [, array $driver_options = array()])

- The prepared statement execution consists of two stages:
  - prepare and
  - execute.
- At the prepare stage, a statement template is sent to the database server.
  - The server performs a syntax check and initializes server internal resources for later use.
- The MySQL server supports using anonymous, positional placeholders with *?*.
- PDO accepts two kinds of parameter markers:
  - named parameter -> :name
  - anonymous parameter -> ?
- You must choose one or the other, they cannot be mixed.

Prepared Statements

- You must include a unique parameter marker for each value you wish to pass into the statement when you call PDOStatement::execute().
- You cannot use a named parameter marker of the same name more than once in a prepared statement.
- Example: prepared statement

```
try {
    $dbh = new PDO("mysql:host=$hostname; dbname=demo", "root", "vertrigo");
}catch(PDOException $e) {
    echo "Error: ".$e->getMessage();
    die();
}
$sql = "DELETE FROM users WHERE id=?";
$pds = $pdo->prepare($sql);
```

Prepared Statements

- If the prepared statement includes parameter markers, you must attach data to them.
- This can be done in two ways:
  - Use PDOStatement::bindValue() or PDOStatement::bindParam() to bind PHP variables to the parameter markers. Bound variables pass their value as input to their associated parameter markers.
    - PDOStatement::bindParam() - is called to bind variables to parameter markers
    - PDOStatement::bindValue() - to bind values to the parameter markers.
  - Pass an array of input-only parameter values
- To execute a prepared statement, we use the execute() method of PDOStatement.

  bool PDOStatement::execute ([array $input_parameters])
- The parameter $input_parameters is an array of values with as many elements as there are bound parameters in the SQL statement executed.
- The array values are bound to parameter markers.

Prepared Statements

❖ Example: using an array to bind a value to a parameter

```
$stmt = $db->prepare("SELECT * FROM users WHERE firstname=? AND lastname=?");
$stmt->execute(array("John", "Mike"));
$rows = $stmt->fetchAll(PDO::FETCH_ASSOC);
```

❖ Example: using unnamed prepared statements

```
$sql = 'INSERT INTO users VALUES(?, ?, ?, ?, ?, ?)';
$stmt = $dbc->prepare($sql);
$stmt->execute(array('RV/3421/06', 'Rosa', ‘Luxembourg ‘, 'female', 09884574, 1))
```

Prepared Statements

- The other way to bind a value to parameter markers is through the PDOStatement::bindValue() method.
  - It binds a value to a corresponding named or question mark placeholder in the SQL statement that was used to prepare the statement.

PDOStatement::bindValue ($parameter , $value [, int $data_type = PDO::PARAM_STR])

- It binds a value to a parameter.
- The parameters are:
  - parameter - it is parameter identifier. For a prepared statement using named placeholders, this will be a parameter name of the form :name. For a prepared statement using question mark placeholders, this will be the 1-indexed position of the parameter.
  - value - the value to bind to the parameter.
  - data_type - explicit data type for the parameter using the PDO::PARAM_* constants.

Prepared Statements

❖ Example: using bindvalue for unnamed parameter markers

```
$sql = 'INSERT INTO users VALUES(?, ?, ?, ?, ?, ?)';
$stmt = $dbc->prepare($sql);
$stmt->bindValue(1, 'RV/3421/06');
$stmt->bindValue (2, 'Rosa');
$stmt->bindValue (3, 'Luxemberg');
$stmt->bindValue (4, 'female');
$stmt->bindValue (5, 09884574, PDO::PARAM_INT);
$stmt->bindValue (6, 1, PDO::PARAM_INT);
$stmt->execute();
if($stmt->rowCount() == 1)
        echo "Inserted";
else
         echo "Not inserted";
```

Prepared Statements

- The other way to bind values to a parameter placeholder is by using PDOStatement::bindParam().
  - This method binds a PHP variable to a corresponding named or anonymous placeholder in the SQL statement.
- Unlike bindValue(), the variable is bound by reference and will only be evaluated at the time that execute() is called.

  PDOStatement::bindParam (mixed $parameter , mixed &$variable);
- It binds a parameter to the specified variable name.
- The parameters are:
  - parameter - parameter identifier. For a prepared statement using named placeholders, this will be a parameter name of the form :name. For a prepared statement using question mark placeholders, this will be the 1-indexed position of the parameter.
  - variable - name of the PHP variable to bind to the SQL statement parameter.

Prepared Statements

❖ Example: binding to ? placeholders

```
$fn = "Chala";
$sx = "male";
$sql = 'select * from users where firstname=? and sex=?';
$stmt = $dbc->prepare($sql);
$stmt->bindParam(1, $fn);
$stmt->bindParam(2, $sx);
$stmt->execute();
```

❖ Example: binding to named placeholders

```
$fn = "Chala";
$sx = "male";
$sql = 'select * from users where firstname=:fname and sex=:fsex';
$stmt = $dbc->prepare($sql);
$stmt->bindParam(":fname", $fn);
$stmt->bindParam(":fsex", $sx);
$stmt->execute();
```

```
while($row = $stmt->fetch(PDO::FETCH_ASSOC)) {
    echo $row['ID'];
    echo $row['firstname'];
    echo $row['lastname'];
    echo $row['sex'];
    echo $row['telephone']."<br>";
}
```

- Prepared statements allow you to execute the same query again and again.
- This simplifies executing queries.

```
# named placeholders
$STH = $dbc->prepare("INSERT INTO address (name, addr, city) values (:name, :addr, :city)");
# assign variables to each placeholder, indexed 1-3
$STH->bindParam(:name,  $name);
$STH->bindParam(:addr,  $addr);
$STH->bindParam(:city,  $city);
# insert one row
$name = "Daniel"
$addr = "Arlington Heights ";
$city = "Berlin";
$STH->execute();
# insert another row
```

Other PDO/PDOStatement Methods

A. Getting Affected Rows

- One of the things we do when executing a query that changes the database (delete, insert, or update) is to check if the query is executed successfully and how many rows are affected by it.
- To do this, we can use PDOStatement's rowCount() method.

```
public int PDOStatement::rowCount()
```

- It returns the number of rows affected by the last DELETE, INSERT, or UPDATE statement executed by the corresponding PDOStatement object.

```
$del = $dbh->prepare('DELETE FROM users where ID=?');
$del->execute(array('RV456/09'));
/* Return number of rows that were deleted */
$count = $del->rowCount();
print("Deleted $count rows.\n");
```

B. Setting Attributes

- To set attributes for PDO, we use PDO::setAttribute().

  PDO::setAttribute (int $attribute , mixed $value)

- This method sets an attribute on the database handle. Some of the available generic attributes are listed below
- *PDO::ATTR_CASE*: force column names to a specific case.
  - *PDO::CASE_LOWER*: force column names to lower case.
  - *PDO::CASE_NATURAL*: leave column names as returned by the database driver.
  - *PDO::CASE_UPPER*: force column names to uppercase.
- *PDO::ATTR_ERRMODE*: error reporting mechanism.
  - *PDO::ERRMODE_SILENT*: just set error codes.
  - *PDO::ERRMODE_WARNING*: raises E_WARNING.
  - *PDO::ERRMODE_EXCEPTION*: throws exceptions.

- *PDO::ATTR_ORACLE_NULLS*: conversion of NULL and empty strings. It is available with all drivers, not just Oracle.
  - *PDO::NULL_NATURAL*: no conversion done.
  - *PDO::NULL_EMPTY_STRING*: empty string is converted to NULL.
  - *PDO::NULL_TO_STRING*: NULL is converted to an empty string.
- *PDO::ATTR_STRINGIFY_FETCHES*: Convert numeric values to strings when fetching. It requires bool value.
- *PDO::ATTR_TIMEOUT*: this specifies the timeout duration in seconds.
  - Not all drivers support this option, and its meaning may differ from driver to driver. For example, sqlite will wait for up to this time value before giving up on obtaining an writable lock, but other drivers may interpret this as a connect or a read timeout interval. It requires int value.
- *PDO::MYSQL_ATTR_USE_BUFFERED_QUERY* (available in MySQL): use buffered queries.
- *PDO::ATTR_DEFAULT_FETCH_MODE*: sets the default fetch mode.
  - The value for this is one of PDO::FETCH_* constants.

Other PDO/PDOStatement Methods…

❖ Example: specify the default fetch mode

```
$connection = new PDO($connection_string, “root”, “vertrigo”);
$connection->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE, PDO::FETCH_OBJ);
```

❖ Example: throw an exception in case of error

```
$connection = new PDO($connection_string, "root", "vertrigo");
$connection->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
```

❖ Example: set the case of column names to lower case

```
$dbConnection = new PDO($dsn, $user, $pass);
$dbConnection->setAttribute(PDO::ATTR_CASE, PDO::CASE_LOWER);
```

❖ Example: attributes can also be passed as an option argument during connection

```
try{
        $db = new PDO('mysql:host=localhost;dbname=demo', 'root', 'vertrigo', array(PDO::ATTR_CASE=>PDO::CASE_UPPER,
                        PDO::ATTR_DEFAULT_FETCH_MODE=>PDO::FETCH_OBJ));
      $stmt = $db->query('SELECT * FROM users');
      while($obj = $stmt->fetch()) {
                        echo $obj->ID;
                        echo $obj->FIRSTNAME;
                        echo $obj->LASTNAME;
                        echo $obj->SEX;
                        echo $obj->TELEPHONE."<br>";
      }
     $db = null;
}catch(PDOException $e) {
                echo "Error: ". $e->getMessage();
}
```

Other PDO/PDOStatement Methods…

C. Setting Fetch Mode

- It is possible to set how the data is fetched from PDOStatement using the setFetchMode() method.
- It sets the default fetch mode for this specific statement.

PDOStatement::setFetchMode(int $mode)

PDOStatement::setFetchMode(int $PDO::FETCH_COLUMN , int $colno)

PDOStatement::setFetchMode(int $PDO::FETCH_CLASS , string $classname , array $ctorargs)

PDOStatement::setFetchMode(int $PDO::FETCH_INTO, object $object)

- The parameters are:
  - mode - the fetch mode must be one of the PDO::FETCH_* constants.
  - colno - column number if you are fetching only one column.
  - classname - class name to fetch the data into.
  - ctorargs - constructor arguments that are passed to constructor of the class.
  - object – Object

Other PDO/PDOStatement Methods…

❖ Example: set fetch mode

```
$sql = "SELECT ID, firstname, lastname, sex FROM users";
try {
   $stmt = $dbh->query($sql);
   $result = $stmt->setFetchMode(PDO::FETCH_NUM);
   while ($row = $stmt->fetch()) {
        print $row[0] . "\t" . $row[1] . "\t" . $row[2] . "\t" . $row[3];
   }
} catch (PDOException $e) {
     print $e->getMessage();
}
```

Other PDO/PDOStatement Methods…

D. Getting Column Count

- To get the number of columns returned from a query execution, use the PDOStatement::columnCount() method.

  int PDOStatement::columnCount()

- It returns the number of columns in the result set represented by the PDOStatement object.
- Example: getting column count

```
$dbh = new PDO($dsn, “root”, “vertrigo”);
$sth = $dbh->prepare("SELECT firstname, lastname, sex FROM users");
$sth->execute();
$colcount = $sth->columnCount();
print("Result set has $colcount columns <br>");
```

Transaction in PDO

- Transactions offer four major features: Atomicity, Consistency, Isolation, and Durability (ACID).
  - Transactional work can also be automatically undone at your request (provided you haven't already committed it), which makes error handling in your scripts easier.
- Unfortunately, not every database supports transactions
  - So PDO needs to run in what is known as "auto-commit" mode when you first open the connection.
- **Auto-commit mode** means that every query that you run has its own implicit transaction if the database supports it, or no transaction if the database doesn't support transactions.

Transaction in PDO…

- If you need a transaction, you must use the PDO::beginTransaction() method to initiate one.
  - If the underlying driver does not support transactions, a PDOException will be thrown.
- The methods used for transactions are: bool PDO::beginTransaction()
- Turns off autocommit mode. While autocommit mode is turned off, changes made to the database via the PDO object instance are not committed until you end the transaction by calling PDO::commit().
- It returns TRUE on success or FALSE on failure. It throws a PDOException if there is already a transaction started or the driver does not support transactions.

  bool PDO::commit()
- Commits a transaction, returning the database connection to autocommit mode until the next call to PDO::beginTransaction() starts a new transaction.
- It returns TRUE on success or FALSE on failure. It throws a PDOException if there is no active transaction.

Transaction in PDO…

- bool PDO::rollBack()
  - It rolls back the current transaction, as initiated by PDO::beginTransaction().
  - If the database was set to autocommit mode, this function will restore autocommit mode after it has rolled back the transaction.
- bool PDO::inTransaction ()
  - It checks if a transaction is currently active within the driver.
  - This method only works for database drivers that support transactions.
- If something goes wrong, the catch block rolls back all changes made since the transaction was started, and then prints out an error message.

```
try {
    $dbh->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    $dbh->beginTransaction();
    $dbh->exec("insert into staff (id, first, last) values (23, 'Joe', 'Bloggs')");
    $dbh->exec("insert into salarychange (id, amount, changedate) values (23, 50000,
NOW())");
    $dbh->commit();
} catch (Exception $e) {
    $dbh->rollBack();
    echo "Transaction failed: " . $e->getMessage();
}
```

PDOException Class

- The PDOException class represents an error raised by PDO.
  - You should not throw a PDOException from your own code.
- The class has a set of properties and methods that allows to access the error that occurred.
- Using them, it is possible to access detailed error information. Some of the methods are:
  - PDOException::getMessage() — gets the Exception message. It returns string.
  - PDOException::getCode() — returns the Exception code. It returns integer error code.
  - PDOException::getLine() — get line number where the exception was created.
  - PDOException::getPrevious()— returns the previous Exception. It returns an Exception object.
  - array PDOException::getTrace ()— returns the Exception stack trace.

PDOException Class

❖ Example: getting error information

```
try {
    $dbh = new PDO($dsn, 'root', 'vertrigo');
} catch (PDOException $e) {
    echo("Error message: " . $e->getMessage());
    echo ("Error code: ". $e->getCode());
}
```